Desafio Full Stack - Tela de Notas

Orientação de Confidencialidade

Classificação: [ ] Público [ ] Interno [ ] Restrito **[X] Confidencial**

Orientação: Este documento e os respectivos materiais deste desafio são confidenciais e destinados apenas a você. Se o recebeu por engano, notifique o remetente e apague-o. É proibido copiar, encaminhar ou usar o conteúdo sem autorização expressa.

Introdução

Seja bem-vindo(a)! Este desafio foi projetado para avaliar de forma integrada as capacidades técnicas de candidatos(as) à vaga de Desenvolvedor(a) de Sistemas, com foco tanto no desenvolvimento **Backend** quanto **Frontend**, independente da senioridade.

O objetivo é construir uma aplicação completa: uma API RESTful (Backend) para gerenciar dados de notas e uma interface web (Frontend) para consumir, exibir e interagir com esses dados.

Instruções Gerais

1. **Estrutura do Projeto:**

    - Crie uma pasta principal em seu computador com o seguinte nome: Seu Nome Completo - Desafio Full Stack.
    - Dentro desta pasta, crie duas subpastas: backend e frontend.
    - Na **raiz da pasta principal**, inicie um único repositório Git (git init). Você usará este repositório para versionar o código de ambas as aplicações.

2. **Versionamento:**

    - Utilize o padrão **Conventional Commits Pattern** para todos os seus commits. Caso não conheça, você pode encontrar mais detalhes neste link.

3. **Documentação:**

    - Crie um arquivo README.md para a documentação principal do seu projeto. Qualquer documentação adicional também deve ser mantida dentro do diretório do projeto.

4. **Entrega:**

    - Ao finalizar, compacte **toda a pasta principal do projeto** (incluindo o .git, backend/ e frontend/) em um único arquivo **.zip**.
    - O nome do arquivo deve seguir o padrão: Seu Nome Completo - Desafio Full Stack.zip.
    - Envie o arquivo .zip para o recrutador através do meio de comunicação que vocês combinaram.

Parte 1: Desafio Backend

Nesta parte, você construirá a API que servirá os dados para o frontend.

Requisitos Obrigatórios (Backend):

- **Tecnologia:** A escolha da linguagem e do framework é livre.
- **Banco de Dados:** Os dados devem ser persistidos em um banco de dados à sua escolha.
- **Carga Inicial de Dados (Seed):** A aplicação deve conter um script ou mecanismo para popular o banco de dados com as informações do arquivo materials/notes.csv. Após essa carga inicial, a API deve interagir **apenas com o banco de dados**.
- **Endpoints:**
    - GET /api/v1/notes: Deve listar todas as notas.
        - Este endpoint deve permitir a **filtragem** via *query parameters*:
            - Por site (ex: /api/v1/notes?site=NomeDoSite).
            - Por equipment (ex: /api/v1/notes?equipment=Gerador).
            - Por um período de data, startDate e endDate (ex: /api/v1/notes?startDate=2024-01-01&endDate=2024-01-31).
    - POST /api/v1/notes: Deve permitir a criação de uma nova nota.
- Implementar **paginação** no endpoint de listagem.
- Adicionar **validação de dados** na criação e atualização de notas.
- Criar uma documentação para a API.

Requisitos Opcionais (Backend):

- Implementar os demais endpoints do **CRUD**.
- **Conteinerizar** a aplicação e o banco de dados com Docker e Docker Compose.
- Escrever **testes automatizados** (unitários e/ou de integração).

Formato do Dado da Nota:

```JSON
{
  "id": "string (uuid)",
  "site": "string",
  "equipment": "string",
  "variable": "string",
  "timestamp": "string (ISO 8601 Date)",
  "author": "string",
  "message": "string"
}
```

Parte 2: Desafio Frontend

Com a API pronta, o próximo passo é construir a interface que irá consumi-la.

Requisitos Obrigatórios (Frontend):

- **Tecnologia:** O conjunto mínimo a ser utilizado é **HTML, CSS e JavaScript (ES6)**. O uso de frameworks (React, Vue, Angular, etc.) é um diferencial, mas não obrigatório.
- **Integração:** A aplicação deve consumir **exclusivamente a API Backend** que você desenvolveu na Parte 1 para buscar, filtrar e exibir os dados. O uso do arquivo materials/notes.csv não é permitido nesta etapa.
- **Interface:**
    - Implemente a interface proposta no arquivo materials/ui-design.pdf.
    - Possua o header lateral (a implementação das outras telas é apenas visual).
    - A tela deve apresentar todas as informações de uma nota: Site, Equipamento, Variável, Data, Autor e Mensagem.
- **Funcionalidades:**
    - Permitir que o usuário filtre as notas por **site** e por **equipamento**.
    - Permitir que o usuário filtre as notas por um **período de data**.
    - As ações de filtro devem realizar novas chamadas à API do backend com os parâmetros corretos.
    - Permitir a **criação de uma nova nota** através de um formulário, consumindo o endpoint POST /api/v1/notes do seu backend.

Requisitos Opcionais (Frontend):

- Permitir **exportar os dados** da tabela para pelo menos um formato (ex: CSV, TXT, PDF).
- Integração com o **CRUD** completo da API, mas lembre-se que este requisito depende da implementação dos endpoints correspondentes no backend.

Critérios de Avaliação

1. **Gerais:**
    - **Funcionalidade e Aderência ao Escopo**: Verificação da entrega e do correto funcionamento de todos os requisitos definidos como obrigatórios, tanto no backend quanto no frontend.
    - **Versionamento:** Qualidade, clareza e granularidade dos commits.
    - **Estrutura do Projeto:** Organização das pastas backend e frontend e clareza geral.
2. **Específicos do Backend:**
    - **Qualidade de Código:** Boas práticas, legibilidade e manutenibilidade.
    - **Design da API:** Aderência aos princípios REST e tratamento de erros.
    - **Persistência de Dados:** Modelagem e uso correto do banco de dados.
3. **Específicos do Frontend:**
    - **Qualidade de Código:** Clareza e organização do código JS, HTML e CSS.
    - **Estrutura do Projeto:** Organização dos componentes e arquivos.
    - **Integração:** Conexão correta e eficiente com a API do backend.

Notas Finais

- **Importante:** O cumprimento dos requisitos não é a única garantia de aprovação. Focamos em avaliar **como** os requisitos foram cumpridos.
- Caso não tenha tido tempo suficiente ou tenha encontrado alguma dificuldade, entregue o desafio ainda que incompleto. Crie uma seção no README.md do projeto explicando quais foram os desafios encontrados. Não se preocupe, avaliaremos ainda assim, pois valorizamos a transparência! :)
- O uso de IA para resolução do código é permitida **desde** que seu uso seja **transparente** e você seja capaz de **explicar** e **justificar** cada linha de código gerada. Encare a IA como uma **ferramenta de auxílio**, não como a autora do projeto. A responsabilidade pela qualidade e funcionalidade do código final é totalmente sua. Além disso, caso utilize IA, adicione em um arquivo chamado prompts.txt os prompts utilizados.
- Está com alguma dúvida ou encontrou um problema no desafio? Entre em contato com a pessoa responsável pelo processo seletivo para esclarecer.

Boa sorte!